

Warehouse Multi-Agent System

Comprehensive Technical Documentation

Deterministic Cooperative Planning | Symmetry Reduction | Formal Verification
Counterexample-Guided Refinement | Real-Time Visualization

MAS Final Project

Python 3.8+ | Pygame | PyYAML

Table of Contents

1. Project Overview & Architecture
2. Configuration System (config.py, config.yaml)
3. Agent Model (agent.py)
4. Environment Engine (env.py)
5. Rendering System (renderer.py)
6. Pathfinding & Planning (pathfinding.py)
7. Symmetry Reduction (symmetry_reduction.py)
8. Formal Verification (verification.py)
9. Counterexample-Guided Refinement (refinement.py)
10. CLI & Orchestration (main.py)
11. Reward Structure & Shaping
12. Test Suite
13. Data Flow & Execution Workflows
14. Algorithm Reference
15. Configuration Parameter Reference

Chapter 1

Project Overview & Architecture

1.1 Purpose

This project implements a multi-agent warehouse simulation where autonomous robots navigate a 2D grid to pick up requested shelves and deliver them to designated goal locations. The system combines cooperative pathfinding, optimal task assignment, symmetry-based state reduction, bounded formal verification, and counterexample-guided constraint refinement into a unified deterministic pipeline.

1.2 Key Capabilities

- Cooperative A* pathfinding with space-time reservation tables (WHCA*-style)
- Hungarian algorithm for minimum-cost robot-to-shelf assignment
- Priority-based planning with rotating priority and carry/idle heuristics
- Role-based symmetry reduction to shrink the verification state space
- Bounded safety verification over quotient (symmetry-reduced) states
- Counterexample-guided refinement: inject constraints when verification fails
- Real-time Pygame visualization with conflict overlays and orbit coloring
- Comprehensive CLI with five operational modes
- Distance-based reward shaping for learning-compatible evaluation

1.3 Technology Stack

- Language: Python 3.8+
- Visualization: Pygame (hardware-accelerated 2D rendering)
- Configuration: PyYAML for external parameter overrides
- No external ML/AI frameworks; all algorithms are hand-implemented

1.4 High-Level Architecture Diagram

The system is organized into nine Python modules with clear dependencies:

```
main.py
+-- config.py      (parameters, YAML loading)
+-- env.py         (grid world, step logic, state vectors)
|   +-- agent.py   (Robot, Direction, Action)
|   +-- renderer.py (Pygame drawing)
+-- pathfinding.py (A*, reservations, assignment, planner)
+-- symmetry_reduction.py (role orbits, canonicalization)
+-- verification.py (bounded quotient verification)
+-- refinement.py  (conflict -> constraint injection)
```

1.5 File Inventory

File	Size	Purpose
config.py	~115 lines	Constants, YAML override loading
config.yaml	~35 lines	External YAML parameter file
agent.py	~168 lines	Robot, Direction, Action classes
env.py	~416 lines	WarehouseEnv grid environment
renderer.py	~185 lines	Pygame rendering (extracted from env)
pathfinding.py	~1284 lines	A*, Hungarian, cooperative planner
symmetry_reduction.py	~88 lines	Role-based orbit detection
verification.py	~100 lines	Bounded safety verification
refinement.py	~90 lines	Constraint injection from conflicts
main.py	~281 lines	CLI entry point, run modes
requirements.txt	2 lines	pygame, pyyaml

Chapter 2

Configuration System

2.1 config.py - Module-Level Constants

All tunable parameters are defined as module-level constants in config.py. This makes them importable by any other module with a simple 'import config as cfg'. The file is organized into four sections: UI Colors, Environment, Planning, and Verification/Refinement.

UI Color Constants

Twelve RGB tuples define the color palette for the Pygame renderer: background, grid lines, robot body/shadow/carrying states, shelf idle/active/shadow, goal primary/shadow, text colors, and an accent gold for indicator dots.

Environment Parameters

Parameter	Default	Description
GRID_W	16	Grid width in cells
GRID_H	16	Grid height in cells
CELL_SIZE	80	Cell size in pixels
NUM_AGENTS	6	Number of robots
NUM_SHELVES	8	Number of shelves
GOALS	[(7,14),(8,14)]	Delivery goal coordinates

Planning Parameters

Parameter	Default	Description
ACTION_SIZE	5	Number of discrete actions
PLAN_HORIZON	30	A* time horizon (steps)
ASTAR_MAX_NODES	3500	Max node expansions per A* call
IDLE_LIMIT	4	Steps before reassigning idle robot
RESERVATION_WINDOW	8	Steps reserved in space-time table
UNPLANNED_HOLD_STEPS	2	Steps to hold unplanned robots still
ESCAPE_IDLE_STEPS	6	Steps before escape heuristic activates
RENDER_FPS	2	Visualization frames per second

Verification / Refinement Parameters

Parameter	Default	Description
MIN_SEPARATION	1	Min Manhattan distance for safety
VERIFY_HORIZON	30	Verification trial length (steps)
VERIFY_TRIALS	20	Number of random-reset verify trials
REFINE_ITERATIONS	2	Max verify-refine loop iterations

REFINE_MAX_CONSTRA	100	Max constraints per refine round
--------------------	-----	----------------------------------

2.2 YAML Override System

The `load_from_yaml()` function reads an external YAML file (default: `config.yaml`) and overrides module-level constants. The mapping from YAML paths to Python globals is defined in the `_YAML_INT_FIELDS` dictionary, which maps (section, key) tuples to global variable names.

The function uses `setattr(module, global_name, int(value))` to dynamically update each constant. After all overrides are applied, `STATE_SIZE` is recomputed via `state_size()`. Goals are handled separately as a list of coordinate tuples.

state_size() Function

Computes the input vector length for one robot's observation:

$$\text{state_size} = 4 + \text{NUM_SHELVES} * 4 + (\text{NUM_AGENTS} - 1) * 4$$

The 4 robot fields are: normalized x, normalized y, direction/3, carrying flag.

Each shelf contributes: normalized x, normalized y, carried flag, requested flag.

Each other robot contributes: normalized x, normalized y, direction/3, carrying flag.

2.3 config.yaml - Sample Configuration

```
grid:
  width: 16
  height: 16
  cell_size: 80

agents:
  num_agents: 6
  num_shelves: 8
  goals:
    - [7, 14]
    - [8, 14]

planning:
  horizon: 30
  astar_max_nodes: 3500
  idle_limit: 4
  reservation_window: 8
  unplanned_hold_steps: 2
  escape_idle_steps: 6

render:
  fps: 3

verification:
  min_separation: 1
  horizon: 30
  trials: 20

refinement:
  iterations: 2
  max_constraints: 100
```

Chapter 3

Agent Model (agent.py)

3.1 Direction Enum

Four cardinal directions encoded as integers 0-3: UP=0, RIGHT=1, DOWN=2, LEFT=3. Movement deltas are stored in module-level arrays MOVE_X and MOVE_Y:

MOVE_X = [0, 1, 0, -1] (dx for UP, RIGHT, DOWN, LEFT)

MOVE_Y = [-1, 0, 1, 0] (dy for UP, RIGHT, DOWN, LEFT)

Origin (0,0) is the top-left corner; y increases downward.

3.2 Action Enum

Five discrete actions available to each robot every time step:

FORWARD (0): Move one cell in the robot's current facing direction

TURN_LEFT (1): Rotate 90 degrees counterclockwise ($\text{dir} - 1 \bmod 4$)

TURN_RIGHT (2): Rotate 90 degrees clockwise ($\text{dir} + 1 \bmod 4$)

PICK_DROP (3): Pick up or drop/deliver a shelf at current position

WAIT (4): Stay in place (incurs small penalty of -0.003)

3.3 Robot Class

Each robot is an instance of the Robot class with the following attributes:

- id (int): Unique identifier
- x, y (int): Grid position
- dir (Direction): Current facing direction (random on init)
- carrying (Optional[Dict]): Reference to carried shelf dict, or None

Movement: forward(env)

Attempts to move one cell forward. Returns (moved: bool, bump: bool).

The method checks three blocking conditions in order:

1. Boundary check: is the target cell inside the grid?
2. Robot collision: is another robot occupying the target cell?
3. Shelf collision via `_occupied_by_shelf()`: complex planner-aware logic.

If not blocked, updates robot position (and carried shelf position if applicable).

Shelf Collision Logic: `_occupied_by_shelf()`

This method implements five rules for shelf-robot interaction:

1. Carried shelves never block (they move with their carrier).
2. If the robot is already carrying a shelf, ALL other shelves block.
3. If the shelf is the robot's planner-assigned target AND is requested, allow entry.

4. Non-requested shelves always block.
5. Requested shelves without planner assignment block.

The planner communicates allowed entries via `env._planner_allowed_shelf_entries`.

Pick or Drop: `pick_or_drop(env)`

Dispatches to one of three internal methods based on state:

- `_pick_shelf_here()`: If not carrying, picks the first matching shelf at current cell.
Reward: +5.0 for requested shelf, +0.0 for non-requested.
- `_deliver_shelf()`: If carrying a requested shelf AND on a goal cell.
Removes the shelf from `env.shelves`, spawns a new requested shelf at a random free position, clears carrying. Reward: +20.0.
- `_drop_shelf()`: If carrying but not at a goal (or non-requested shelf).
Marks shelf as not carried, clears carrying. Reward: -0.1.

Chapter 4

Environment Engine (env.py)

4.1 WarehouseEnv Class Overview

WarehouseEnv is the central simulation engine. It manages the grid world, robot and shelf placement, action execution, collision detection, reward computation, and state vector generation. It delegates rendering to a separate Renderer instance.

4.2 Initialization

Constructor parameters:

- render (bool): Enable/disable Pygame visualization
- num_agents (Optional[int]): Override agent count from config

On init, the environment computes an optimal cell size for the display, creates the Renderer if rendering is enabled, and calls reset().

4.3 reset()

Resets the environment to a fresh random state:

1. Goals are marked as occupied (no spawning on goal cells).
2. Robots are spawned at random free positions with random directions.
3. Shelves are spawned at random free positions, all marked requested=True.
4. All counters, heat maps, event logs, and planner state are cleared.
5. Returns a list of state vectors, one per robot.

4.4 step(actions) - The Core Loop

The step function processes one time step. It accepts a list of integer-encoded actions (one per robot) and returns (next_states, rewards, done, collisions, trajectories).

Phase 1: Decode & Record

Actions are decoded from integers to Action enums. Missing actions default to WAIT. Intent conflicts are pre-recorded for visualization via _record_intent_conflicts().

Phase 2: Non-Forward Actions

Turn and pick/drop actions are processed immediately:

- TURN_LEFT / TURN_RIGHT: Rotate robot, reward penalty -0.002
- PICK_DROP: Delegate to robot.pick_or_drop(), accumulate reward
- WAIT: Reward penalty -0.003

Forward intents are collected but not yet executed.

Phase 3: Forward Conflict Resolution

Forward intents pass through `_collect_forward_intent()` which checks boundary and shelf blocking. Valid intents are then resolved by the static method `_resolve_forward_conflicts()` which detects three types of conflicts:

1. Static conflicts: target cell occupied by a non-moving robot.
2. Same-target conflicts: multiple robots want the same cell.
3. Head-on swap conflicts: two robots trying to swap positions.

Blocked robots receive a -0.2 collision penalty. Successful movers get +0.01.

Phase 4: Reward Shaping

After movement, `_apply_distance_rewards()` computes reward based on distance change to each robot's target:

- Getting closer: $+0.12 * \text{distance_reduction}$
- Getting farther: $-0.04 * \text{distance_increase}$
- Carrying non-requested shelf: -0.05 per step

If any deliveries occurred, a team bonus of $2.0 * \text{delivery_count}$ is added to ALL robots.

Phase 5: Bookkeeping

Traffic and collision heat maps are updated. Step counter increments. Episode terminates after 1000 steps (`done=True`). Event log is maintained (last 28 entries). Final state vectors are computed for all robots.

4.5 State Vector: `get_state(robot)`

Each robot receives a normalized observation vector of size `STATE_SIZE`:

- [robot_x/W, robot_y/H, dir/3, carrying] +
- [shelf_x/W, shelf_y/H, carried, requested] * `NUM_SHELVES` +
- [other_x/W, other_y/H, dir/3, carrying] * (`NUM_AGENTS` - 1)

All values are in [0, 1]. Shelves are sorted by ID for consistency.

4.6 `evaluate(planner, ...)`

Runs multiple episodes with the given planner and returns the average collision rate per agent per episode. Used for benchmarking planner quality.

4.7 UI Event Handling

`handle_event()` processes Pygame events for agent selection:

- Mouse click on a robot: select that robot
- Tab / Right / Period: cycle to next robot
- Left / Comma: cycle to previous robot

4.8 Conflict Recording: `_record_intent_conflicts()`

Before resolving conflicts, this method pre-computes intent-level conflicts for visualization. It detects:

- Boundary conflicts: robot trying to exit the grid
- Vertex conflicts: multiple robots targeting the same cell

- Edge (swap) conflicts: two robots swapping positions

Results are stored in `env.last_conflicts` for the renderer's overlay system.

Chapter 5

Rendering System (renderer.py)

5.1 Design

The Renderer class was extracted from env.py to separate visualization concerns from simulation logic. It holds Pygame surface references, fonts, and drawing methods. The environment calls `renderer.draw(env, fps)` each frame.

5.2 Drawing Pipeline

Each frame executes these drawing layers in order:

- Screen fill with BG_PURE (white)
- `_draw_grid()`: Anti-aliased grid lines
- `_draw_goals()`: Blue rectangles with 'GOAL' label
- `_draw_shelves()`: Green (requested) or gray (idle) rectangles with shelf ID
- `_draw_robots()`: Circles with directional arrows, orbit color indicators
- `_draw_conflicts()`: Red overlays for vertex conflicts, red lines for edge conflicts
- `pygame.display.flip()` + `clock.tick(fps)`

5.3 Robot Rendering Details

Robots are drawn as filled circles with anti-aliased edges. Carrying robots use an orange color (ROBOT_CARRYING); normal robots use blue (ROBOT_PRIMARY). A gold indicator dot appears above robots carrying requested shelves.

Directional arrows are rendered with a rectangular shaft and a filled triangular arrowhead pointing in the robot's facing direction.

Each robot has a small colored square indicating its symmetry orbit, obtained from `_orbit_map()` which calls `symmetry_reduction.detect_role_orbits()`.

5.4 Conflict Overlay

Vertex conflicts are drawn as semi-transparent red rectangles over the affected cell. Edge (swap) conflicts are drawn as red lines between the two swapping cells. These overlays use the `env.last_conflicts` list populated by `_record_intent_conflicts()`.

Chapter 6

Pathfinding & Planning (pathfinding.py)

6.1 Module Overview

At 1,284 lines, pathfinding.py is the largest and most complex module. It implements the entire cooperative planning pipeline: time-expanded A* search, space-time reservation tables, constraint tables, Hungarian optimal assignment, and the cooperative planner that orchestrates everything.

The code is written using extremely simple, explicit Python syntax: no list comprehensions, no any()/all() builtins, no lambda expressions, no ternary operators, and no enumerate(). Every idea is expressed in one clear line.

6.2 Data Structures

NodeKey (frozen dataclass)

Immutable key for A* search nodes in the time-expanded graph:

(x: int, y: int, dir_value: int, t: int)

Frozen to enable use as dictionary keys and heap elements.

_TimedTable

Base class for time-indexed position and edge lookup tables. Uses defaultdict(set) for O(1) average-case membership checks. Provides has_position() and has_edge() methods.

ReservationTable (extends _TimedTable)

Records which (position, time) and (edge, time) cells are reserved by already-planned robots. Methods:

- reserve_position(pos, t): Mark cell as occupied at time t
- reserve_edge(from, to, t): Mark edge traversal at time t
- reserve_positions(path): Reserve an entire path (positions + edges)
- is_reserved(pos, t): Check if cell is taken at time t
- is_edge_reserved(from, to, t): Check if edge is taken at time t

ConstraintTable (extends _TimedTable)

Records forbidden (position, time) and (edge, time) pairs injected by the refinement module. Methods mirror ReservationTable but use 'forbid' and 'is_forbidden' naming to distinguish from reservations.

6.3 Utility Functions

manhattan(a, b)

Returns the Manhattan distance between two grid positions: $|ax-bx| + |ay-by|$.

apply_action(x, y, direction, action, grid_w, grid_h)

Computes the result of applying an action to a robot state. Returns (new_x, new_y, new_direction) or None if the action moves out of bounds. Used by both A* and simulation.

simulate_positions(start, dir, actions, w, h, horizon)

Simulates a sequence of actions and returns the list of positions at each time step. Used to populate reservation tables after planning.

_blocked_by_constraints(constraints, cur, next, t)

Checks if a move from cur to next at time t is forbidden by any constraint table in the given iterable. Returns True if forbidden.

6.4 Hungarian Algorithm: minimum_cost_matching()

A full implementation of the Hungarian (Kuhn-Munkres) algorithm for optimal bipartite matching. Solves the assignment problem in $O(n^3)$ time.

Input: costs[i][j] = cost of assigning row i to column j.

Output: assignment[i] = column assigned to row i, or -1 if unassigned.

Implementation details:

- Rectangular matrices are padded to square with a large pad cost.
- Uses dual variables (u, v potentials) for augmentation.
- The algorithm iterates over rows, maintaining minimum slack values.
- Augmenting paths are found and applied to update the assignment.

Used in two places:

1. AssignmentManager.update_assignments(): robot-to-shelf matching
2. CooperativePlanner._assign_delivery_goals(): carrier-to-goal matching

6.5 A* Time-Expanded Search: astar_time()

The core pathfinding function. Searches a 4D space-time graph where each node is (x, y, direction, time_step). The search expands nodes using a priority queue ordered by f-score = g-score + manhattan heuristic.

Parameters

start_pos: Starting (x, y) position

start_dir: Starting Direction enum

goal: Target (x, y) position

grid_w, grid_h: Grid dimensions

reservations: ReservationTable of other robots' plans

max_time: Maximum time horizon for search

blocked_t1: Set of cells blocked at time step 1

blocked_by_time: Dict[t, Set[pos]] of time-specific blocks

blocked_static: Set of permanently blocked cells (shelves)

constraints: Iterable of ConstraintTables (from refinement)

max_expansions: Maximum node expansions before giving up

Search Procedure

1. Initialize open list with start node (f = manhattan to goal).
2. Pop node with lowest f-score from the priority queue.
3. If at goal position, reconstruct path by following parent links.
4. If time exceeds max_time or expansions exceed max_expansions, skip/return None.
5. For each action in ACTION_ORDER (FORWARD, TURN_LEFT, TURN_RIGHT, WAIT):
 - a. Apply action to get candidate (x, y, dir).
 - b. Check all blocking conditions: static blocks, t1 blocks, time-specific blocks, reservations, edge reservations, constraints.
 - c. Compute g-score using ACTION_COSTS (WAIT costs 1.25, others cost 1.0).
 - d. If this path to the candidate is better than known, record and push to heap.
6. If heap empties without reaching goal, return None.

Action Costs

Action	Cost	Rationale
FORWARD	1.0	Standard movement cost
TURN_LEFT	1.0	Rotation cost
TURN_RIGHT	1.0	Rotation cost
WAIT	1.25	Slight penalty to discourage waiting

6.6 AssignmentManager

Manages the mapping between robots and shelves using two bidirectional dictionaries: agent_to_shelf and shelf_to_agent.

update_assignments(env)

1. Remove stale assignments (shelf no longer exists, is carried, or not requested).
2. Find free robots (not carrying, not assigned) and free shelves (requested, not carried, not assigned).
3. Build a cost matrix of Manhattan distances from free robots to free shelves.
4. Run minimum_cost_matching() to find the optimal assignment.
5. Apply the new assignments to both dictionaries.

get_target_for_robot(robot, env, delivery_targets)

Returns the target position for a robot based on its current state:

- Carrying a requested shelf: delivery target (from Hungarian matching) or nearest goal.
- Carrying a non-requested shelf: None (no target).
- Not carrying, has assignment: shelf position.
- Not carrying, no assignment: None.

6.7 IdleTracker

Monitors how long each robot has been stationary. If a robot exceeds `idle_limit` steps without moving, its shelf assignment is dropped (freeing it for reassignment). This prevents deadlocks where a robot is assigned to a shelf it cannot reach.

6.8 CooperativePlanner - Main Orchestrator

The `CooperativePlanner` class coordinates all planning components. It maintains persistent state across time steps: assignments, constraints, idle tracking, and priority rotation.

Constructor Parameters (from config)

- grid_w, grid_h:** Grid dimensions
- plan_horizon:** A* search time horizon (default: 30)
- astar_max_nodes:** Max A* expansions (default: 3500)
- reservation_window:** Steps to reserve (clamped to [2, horizon])
- unplanned_hold_steps:** Steps to block unplanned robot cells (min: 1)
- escape_idle_steps:** Steps before escape heuristic (min: 2)

compute_actions(env) - Main Entry Point

Called every time step. Returns a list of integer-encoded actions, one per robot.

Execution flow:

1. Track idle agents (may drop stale assignments).
2. Update shelf-to-robot assignments via Hungarian matching.
3. Compute delivery targets for carrier robots (tiered Hungarian).
4. Determine planning order via `_planning_order()`.
5. For each robot in priority order:
 - a. Compute blocked cells: other robots at $t=1$, unplanned robots for $t=[2..hold]$.
 - b. Check `_should_pick_drop()`: if on target shelf or on goal with requested shelf.
 - c. Otherwise, call `_plan_navigation()` for A* path planning.
 - d. Commit plan: record action, reserve space-time cells.
6. Store allowed shelf entries for the environment.
7. Sanitize immediate conflicts via `_sanitize_immediate_conflicts()`.
8. Update debug info and return final action list.

_planning_order(robots)

Determines the order in which robots are planned. Uses a three-level sort key:

1. Carry priority: robots carrying requested shelves plan first (0 vs 1).
2. Idle duration: more-idle robots get higher priority (negative for ascending sort).
3. Rotation position: a rotating offset ensures different robots lead each round.

This prevents starvation and prioritizes time-sensitive deliveries.

_plan_navigation(robot, env, ...)

Core navigation decision for a single robot:

1. Get target from AssignmentManager.
2. If no target:
 - a. If idle on a goal cell, try to clear by moving toward nearest shelf.
 - b. Otherwise, WAIT.
3. If has target, run `astar_time()` to find a path.
4. If A* succeeds:
 - a. If path starts with WAIT but robot is idle too long, try escape heuristic.
 - b. Otherwise, follow the A* path.
5. If A* fails:
 - a. If at target, WAIT.
 - b. If idle too long, try escape heuristic.
 - c. Otherwise, WAIT.

`_best_immediate_action(robot, target, reservations, blocked)`

Escape heuristic: evaluates all four actions + wait and picks the one with the lowest score that is not blocked by reservations, constraints, or static obstacles. Scoring uses manhattan distance to target + action-specific penalties.

`_sanitize_immediate_conflicts(env, actions, priority)`

Post-planning safety check. Reviews all forward-moving robots and resolves:

- Shelf blocking: robots moving into blocked shelf cells are forced to WAIT.
- Same-target conflicts: first robot by priority wins, others WAIT.
- Head-on swap conflicts: lower-priority robot WAITs.

This is a last-resort fix because A* plans are made sequentially and may conflict.

`_assign_delivery_goals(env)`

Assigns specific goal positions to carrier robots using tiered Hungarian matching.

Each goal appears once per 'tier', where tier adds a +2.0 cost penalty. This spreads multiple carriers across different goals rather than all targeting the nearest one.

`_should_pick_drop(robot, env)`

Returns True if the robot should execute PICK_DROP this step:

- Carrying a requested shelf AND on a goal cell -> drop/deliver.
- Not carrying AND standing on assigned, requested shelf -> pick up.

`_blocked_shelf_positions(robot, target, env)`

Returns the set of shelf cells this robot must not enter. All non-carried shelf positions are blocked, EXCEPT the robot's target shelf (when going to pick it up).

Chapter 7

Symmetry Reduction (symmetry_reduction.py)

7.1 Motivation

In multi-agent systems, the state space grows combinatorially with the number of agents. If two robots have the same role (e.g., both idle and not carrying), swapping them produces a structurally equivalent state. Symmetry reduction exploits this by grouping equivalent states into orbits, reducing the verification search space.

7.2 Role Definition: `_agent_role(robot)`

Each robot's role is a 2-tuple:

(carrying_anything: 0 or 1, carrying_requested: 0 or 1)

This yields three possible roles:

(0, 0) = idle, not carrying

(1, 0) = carrying a non-requested shelf

(1, 1) = carrying a requested shelf

7.3 `detect_role_orbits(robots)`

Groups robot indices by role. Returns a list of lists, where each inner list contains the indices of robots sharing the same role. For example, with 6 robots where 2 are carrying requested shelves and 4 are idle:

```
[[0, 1, 3, 5], [2, 4]]
```

7.4 Canonicalization

`canonicalize_agents(robots, orbits)`

Creates a canonical state key by sorting agent states within each orbit. Each agent's state is a 4-tuple: (x, y, dir_value, carrying_requested). Within an orbit, members are sorted lexicographically. The result is a tuple of orbit tuples.

`canonicalize_state(env, include_shelves)`

Constructs the full quotient state key:

1. Agent key: canonicalized agent states within orbits.
2. Shelf key (optional): sorted (x, y, requested) tuples for non-carried shelves.

Used by the verification module to detect state revisitation.

7.5 `build_quotient_model(env)`

Returns a dictionary with:

- orbits: list of robot index groups

- representatives: first index from each orbit
- mapping: agent_index -> orbit_index lookup

Used by the renderer for orbit-based coloring and by the CLI for symmetry detection.

Chapter 8

Formal Verification (verification.py)

8.1 Approach: Bounded Model Checking

The verification module performs bounded checking of the planner's safety properties. It runs multiple trials (each starting from a random initial state) and checks for collisions and minimum separation violations within a fixed time horizon.

The search is performed over symmetry-reduced (quotient) states: if a canonical state has been visited before in the same trial, the trial terminates early (state convergence).

8.2 Safety Properties Checked

- No collisions: `env.last_conflicts` is empty and `collisions == 0`
- Minimum separation: pairwise Manhattan distance $\geq$ `min_separation` at all times

8.3 `verify_on_quotient()` - Main Function

Parameters: `env`, `planner`, `horizon`, `trials`, `include_shelves`, `min_separation`, `progress_every`.

Algorithm:

For each trial (random reset):

1. Check initial separation (pre-step safety).
2. For each step up to horizon:
 - a. Compute canonical state key; if visited, break (convergence).
 - b. Run planner to get actions, execute step.
 - c. If collisions > 0 or conflicts detected: return UNSAFE with counterexample.
 - d. If min distance $<$ `min_separation`: return UNSAFE with separation violation.
 - e. Track overall_min_margin (`delta_q = min_distance - min_separation`).

If all trials pass: return SAFE with `delta_q` and average collision rate.

8.4 Return Value

The function returns a dictionary with keys:

- `safe` (bool): True if all trials passed
- `delta_q` (float): minimum safety margin observed (distance - separation)
- `avg_collision_rate` (float): collisions per step across all trials
- `counterexample` (list): position trace leading to the violation (if unsafe)
- `actions` (list): action history of the counterexample (if unsafe)
- `conflicts` (list): conflict descriptors from the violation (if unsafe)

8.5 `_min_pairwise_manhattan(robots)`

Computes the minimum Manhattan distance between any pair of robots. Returns 999 if fewer than 2 robots exist. $O(n^2)$ complexity.

Chapter 9

Counterexample-Guided Refinement (refinement.py)

9.1 Concept

When verification finds a safety violation, the counterexample (trace + conflicts) is used to inject constraints into the planner's ConstraintTable. These constraints forbid specific (position, time) or (edge, time) combinations, forcing the planner to find alternative paths in future planning rounds.

This is analogous to Counterexample-Guided Abstraction Refinement (CEGAR) in model checking.

9.2 refine_planner_with_conflicts()

Parameters: planner, conflicts, trace, max_constraints.

Processing rules per conflict type:

- vertex: Forbid pos at time+1 in the constraint table. (+1 constraint)
- edge: Forbid both directions of the edge at time+1. (+2 constraints)
- boundary: Forbid the boundary cell at time+1. (+1 constraint)
- separation: Skipped (no actionable constraint).
- unknown: Skipped.

Stops early if max_constraints is reached.

9.3 Fallback: _add_fallback_trace_constraint()

When no explicit conflicts are provided but a trace exists, this fallback:

1. First pass: scans each time step for same-cell collisions.
If found, forbids that (position, time). Returns 1.
2. Second pass: if no collision, finds the closest pair at t=1
and forbids one of them. Returns 1.
3. If trace is too short: returns 0.

Chapter 10

CLI & Orchestration (main.py)

10.1 Entry Point

main.py is the CLI entry point, invoked as 'python main.py [options]'. It uses argparse to parse 18 command-line flags, loads YAML config overrides, sets random seeds, and dispatches to one of five operational modes.

10.2 Operational Modes

interactive (default)

Opens a Pygame window. Each frame: handle UI events, compute planner actions, step the environment, render. Runs indefinitely until Quit or 'Q' key. Resets on episode end.

simulate

Runs multiple episodes with optional rendering. Logs average collision rate at configurable intervals. Supports rendering for visual debugging.

eval

Headless evaluation: runs env.evaluate() over multiple episodes and reports the average collision rate per agent per episode.

verify-refine (--verify-refine flag)

Runs the verification-refinement loop:

1. Run verify_on_quotient() with configured horizon and trials.
2. If SAFE: log result, return.
3. If UNSAFE: apply refine_planner_with_conflicts().
4. Repeat up to refine_iterations times.
5. Log final constraint summary.

detect-symmetry (--detect-symmetry flag)

Creates a fresh environment, runs build_quotient_model(), and prints the detected orbits and quotient summary. Useful for inspecting the symmetry structure.

10.3 CLI Arguments Reference

Argument	Description	Default
--mode	interactive simulate eval	interactive
--render	Enable Pygame rendering	off
--config	Path to YAML config	config.yaml

--seed	Random seed	None
--cell-size	Override cell size (px)	from config
--episodes	Simulation episodes	8
--steps-per-episode	Max steps per episode	200
--log-interval	Log every N episodes	1
--eval-episodes	Evaluation episodes	3
--plan-horizon	A* time horizon	from config
--detect-symmetry	Run symmetry detection	off
--verify-refine	Run verify-refine loop	off
--verify-horizon	Verification horizon	from config
--verify-trials	Verification trials	from config
--verify-include-shelves	Include shelves in quotient	off
--verify-progress	Progress update frequency	1
--min-separation	Min Manhattan separation	from config
--refine-iterations	Max refine iterations	from config
--refine-max-constraints	Max constraints per round	from config

10.4 _DEFAULTS Dictionary

Maps CLI attribute names to config module constants. When a CLI argument is None (not provided), the config default is used. Applied by `_apply_defaults()`.

Chapter 11

Reward Structure & Shaping

11.1 Per-Step Base Rewards

Action	Reward	Source
FORWARD (success)	+0.01	env.step()
FORWARD (blocked)	-0.20	env.step()
TURN_LEFT/RIGHT	-0.002	env.step()
WAIT	-0.003	env.step()
PICK (requested)	+5.00	agent.pick_or_drop()
PICK (non-requested)	+0.00	agent.pick_or_drop()
DELIVER (at goal)	+20.00	agent.pick_or_drop()
DROP (not at goal)	-0.10	agent.pick_or_drop()
NOOP (no shelf)	-0.05	agent.pick_or_drop()

11.2 Distance-Based Shaping

After each step, the environment computes the change in Manhattan distance to each robot's target:

- Moving closer: $+0.12 * (\text{old_dist} - \text{new_dist})$
- Moving farther: $-0.04 * (\text{new_dist} - \text{old_dist})$
- Carrying non-requested shelf: -0.05 penalty per step

Target is determined by robot state:

- Carrying requested shelf: nearest goal
- Not carrying: nearest requested, uncarried shelf

11.3 Team Delivery Bonus

When one or more deliveries occur in a step, ALL robots receive a bonus of:

$$\text{team_bonus} = 2.0 * \text{number_of_deliveries}$$

This encourages cooperative behavior by rewarding the entire team for each delivery.

11.4 Base Step Penalty

Every robot receives a base penalty of -0.01 per step, regardless of action. Combined with action-specific penalties (WAIT: -0.003, TURN: -0.002), this creates a gradient that encourages productive movement.

Chapter 12

Test Suite

12.1 Test Runner (tests/run_tests.py)

The main test runner uses importlib to dynamically import test modules and discovers test functions by name prefix 'test_'. It runs five core suites and reports pass/fail counts. Two additional suites (test_refinement, test_verification_refinement) are run as standalone scripts.

12.2 Test Suites Overview

test_rewards.py (Reward Structure Tests)

Validates all reward signals:

- test_step_penalty: base step cost is between -0.01 and -0.003
- test_collision_penalty: collision penalty is at least -0.2
- test_delivery_reward: delivery gives > 20.0 (includes team bonus)
- test_pick_reward_requested: picking requested shelf gives 5.0
- test_pick_reward_non_requested: picking non-requested shelf gives 0.0
- test_distance_reward: getting closer increases reward
- test_forward_bonus: successful forward gives +0.01

test_env.py (Environment Tests)

Validates environment mechanics (9 tests):

- test_env_reset: correct state/robot/shelf counts, bounds checking
- test_env_step: action processing returns correct tuple structure
- test_env_state_shape: state vector matches expected STATE_SIZE
- test_env_multiple_resets: 5 sequential resets work correctly
- test_env_collision_detection: forward into occupied cell is blocked
- test_env_swap_collision_detection: head-on swaps produce >= 2 collisions
- test_env_short_action_list_defaults_to_wait: missing actions default to WAIT
- test_env_blocks_unassigned_shelf_entry: robot blocked from unassigned shelf cell
- test_env_goal_delivery: carry requested shelf to goal gives reward > 15.0

test_agent.py (Agent Tests)

Validates Robot class behavior (9 tests):

- Initialization: correct ID, position, direction, carrying=None
- Turn left/right: direction cycles correctly (mod 4 arithmetic)
- Forward movement: moves in facing direction, updates position
- Boundary collision: blocked at grid edge, bump=True
- Agent collision: blocked by another robot, bump=True
- Pick shelf: carries shelf, shelf marked carried, reward >= 0

- Deliver shelf: at goal with requested shelf, reward > 15.0
- Drop shelf: not at goal, penalty <= 0, carrying cleared

test_training.py (Planning Component Tests)

Validates planning components (9 tests):

- test_planner_action_output: correct action count and value range
- test_astar_simple_path: finds path with >= 2 forwards
- test_planner_avoids_head_on_collision: no collision in head-on scenario
- test_planner_waits_when_no_requested_shelves: all WAIT when no tasks
- test_astar_respects_timed_cell_reservations: avoids reserved (cell, time)
- test_planner_waits_when_spacetime_search_fails: WAIT on A* failure
- test_astar_blocks_optimistic_wait_on_unplanned_agent: turns to escape
- test_planner_clears_blocked_goal_cell: robot clears goal for deliveries
- test_planner_avoids_long_global_stall_seed_20: max stall <= 12 steps

test_integration.py (Integration Tests)

End-to-end validation (3 tests):

- test_config_loading: all config parameters are positive
- test_simulation_episode: 50-step episode runs without error
- test_evaluation_functionality: evaluate() returns valid numeric rate

test_refinement.py (Refinement Logic Tests)

Validates constraint injection (10 tests):

- Vertex conflict: adds 1 constraint, position forbidden at t+1
- Edge conflict: adds 2 constraints (bidirectional), edges forbidden at t+1
- Boundary conflict: adds 1 constraint, position forbidden at t+1
- Separation conflict: skipped (0 constraints)
- Multiple conflicts: correct total (vertex + 2*edge + boundary)
- Max constraints limit: stops at exactly max_constraints
- Fallback trace collision: finds same-cell collision, adds 1 constraint
- Fallback trace no collision: finds closest pair, adds 1 constraint
- No conflicts no trace: 0 constraints
- Missing conflict fields: skipped gracefully (0 constraints)

test_verification_refinement.py (Integration)

End-to-end verification-refinement loop test:

1. Run verify_on_quotient() with 15-step horizon, 10 trials.
2. If UNSAFE: apply refinement, then re-verify.
3. Assert: refinement improves safety OR maintains delta_q margin.
4. If already SAFE: confirm no conflicts detected.

Chapter 13

Data Flow & Execution Workflows

13.1 Main Simulation Loop

```
main.py
1. Parse CLI args, load YAML config
2. Create WarehouseEnv(render=True/False)
3. Create CooperativePlanner(grid_w, grid_h, plan_horizon)
4. Loop:
  a. planner.compute_actions(env) -> action list
  b. env.step(actions) -> next_states, rewards, done, collisions
  c. env.render() (if enabled)
  d. If done: env.reset()
```

13.2 Planning Pipeline (per step)

```
compute_actions(env):
1. idle_tracker.track_idle_agents()
2. assignment_manager.update_assignments() [Hungarian]
3. _assign_delivery_goals() [tiered Hungarian]
4. _planning_order() [carry > idle > rotation]
5. For each robot in order:
  a. Compute blocked cells (t=1 + unplanned holds)
  b. _should_pick_drop() -> PICK_DROP or navigate
  c. _plan_navigation() -> astar_time() or escape
  d. _commit_plan() -> reserve space-time cells
6. _sanitize_immediate_conflicts() [final safety pass]
```

13.3 Verification-Refinement Loop

```
run_verify_refine():
  For each iteration (1..refine_iterations):
    1. verify_on_quotient(env, planner, horizon, trials)
      -> For each trial (random reset):
        - Canonicalize state (symmetry reduction)
        - Planner computes actions, env steps
        - Check: collisions? separation? conflicts?
        - If unsafe: return counterexample
    2. If SAFE: done
    3. If UNSAFE: refine_planner_with_conflicts()
      -> Inject vertex/edge/boundary constraints
      -> Or fallback: inject from trace
```

13.4 Shelf Lifecycle

1. Spawn: env.reset() creates shelves at random free positions
All shelves start as requested=True, carried=False
2. Assignment: AssignmentManager pairs robots with shelves

Uses Hungarian algorithm for optimal matching

3. Navigation: Robot plans path to assigned shelf
A* with space-time reservations and constraints
4. Pick up: Robot arrives at shelf, executes PICK_DROP
shelf.carried=True, robot.carrying=shelf, reward=+5.0
5. Delivery: Robot navigates to nearest goal
Tiered Hungarian matching for goal assignment
6. Drop off: Robot at goal executes PICK_DROP
Shelf removed, new shelf spawned, reward=+20.0
7. Respawn: New shelf at random free position
requested=True, ready for next assignment cycle

Chapter 14

Algorithm Reference

14.1 Cooperative A* (WHCA*-Style)

Windowed Hierarchical Cooperative A* (WHCA*) plans for one agent at a time in a shared space-time graph. Each agent's A* search avoids cells already reserved by previously-planned agents. The 'windowed' aspect means only RESERVATION_WINDOW time steps are reserved, reducing interference between distant future positions.

Key differences from standard A*:

- Search space is 4D: (x, y, direction, time)
- Cost function includes action-specific costs (WAIT costs more)
- Blocking checks include reservations, constraints, static obstacles, and time-specific blocks
- Max node expansion limit prevents runaway search on congested grids

14.2 Hungarian Algorithm (Kuhn-Munkres)

Optimal bipartite matching in $O(n^3)$. The implementation:

1. Pads rectangular input to square matrix.
2. Maintains dual variables (u potentials for rows, v for columns).
3. For each row: finds an augmenting path through the bipartite graph.
4. Updates potentials until assignment is found.
5. Reads out assignment from the final matching.

Used for two distinct matching problems: robot-to-shelf (pickup) and carrier-to-goal (delivery).

14.3 Role-Based Symmetry Reduction

Exploits the observation that robots with identical roles are interchangeable:

1. Classify robots by role: (carrying, carrying_requested).
2. Group same-role robots into orbits.
3. Within each orbit, sort agent states lexicographically.
4. The sorted representation is the canonical (quotient) state.

Reduction factor: if k robots share a role, k! states collapse to 1.

For 6 agents with 4 idle + 2 carrying: $4! * 2! = 48x$ reduction.

14.4 Bounded Model Checking

Verifies safety properties within a bounded horizon:

- Multiple trials with random initial states provide coverage.

- Symmetry reduction detects state revisitation (early termination).
- Safety margin (δ_q) quantifies how close to violation.
- Counterexample extraction enables targeted refinement.

Not exhaustive (unlike full model checking), but practical for systems with large state spaces where exhaustive verification is intractable.

14.5 Counterexample-Guided Refinement (CEGAR-Inspired)

Iteratively improves planner safety:

1. Verify: run bounded model checking.
2. If safe: done.
3. If unsafe: extract conflicts from counterexample.
4. Refine: inject time-indexed constraints into planner.
5. Repeat.

Convergence is not guaranteed (bounded checking may miss violations), but each iteration strictly increases the constraint set.

Chapter 15

Configuration Parameter Reference

15.1 Complete Parameter Table

All configurable parameters in one place, with types, defaults, and valid ranges.

Parameter	Type	Default	Range	Description
GRID_W	int	16	1-100	Grid width in cells
GRID_H	int	16	1-100	Grid height in cells
CELL_SIZE	int	80	20-200	Pixel size per cell
NUM_AGENTS	int	6	1-20	Number of robots
NUM_SHELVES	int	8	1-50	Number of shelves
ACTION_SIZE	int	5	5 (fixed)	Discrete action count
PLAN_HORIZON	int	30	5-100	A* time limit
ASTAR_MAX_NODES	int	3500	100-50000	A* expansion limit
IDLE_LIMIT	int	4	1-20	Idle steps before drop
RESERVATION_WINDOW	Wint	8	2-horizon	Reservation depth
UNPLANNED_HOLD	int	2	1-10	Hold unplanned robots
ESCAPE_IDLE	int	6	2-20	Escape trigger threshold
RENDER_FPS	int	2	1-60	Viz frame rate
MIN_SEPARATION	int	1	0-5	Safety min distance
VERIFY_HORIZON	int	30	5-200	Verify trial length
VERIFY_TRIALS	int	20	1-1000	Verify random trials
REFINE_ITERATIONS	int	2	1-20	Refinement rounds
REFINE_MAX_CONST	int	100	1-10000	Max constraints

15.2 YAML Path Mapping

The `_YAML_INT_FIELDS` dictionary maps (YAML_section, YAML_key) tuples to Python global constant names. This enables clean separation between the YAML schema and the Python constant names.

YAML Path	Python Constant
grid.width	GRID_W
grid.height	GRID_H
grid.cell_size	CELL_SIZE
agents.num_agents	NUM_AGENTS
agents.num_shelves	NUM_SHELVES
planning.horizon	PLAN_HORIZON
planning.astar_max_nodes	ASTAR_MAX_NODES
planning.idle_limit	IDLE_LIMIT

planning.reservation_window	RESERVATION_WINDOW
planning.unplanned_hold_steps	UNPLANNED_HOLD_STEPS
planning.escape_idle_steps	ESCAPE_IDLE_STEPS
render.fps	RENDER_FPS
verification.min_separation	MIN_SEPARATION
verification.horizon	VERIFY_HORIZON
verification.trials	VERIFY_TRIALS
refinement.iterations	REFINE_ITERATIONS
refinement.max_constraints	REFINE_MAX_CONSTRAINTS